

StreamBed: Adaptable Production-Ready Distributed Inference and Management System

Ashish Basetty

University of California, Los Angeles

Master of Science, Computer Science Capstone

June 12, 2026

Extended from project report written in collaboration with:

Ivan Sit, Frederick Zhang, and Vishnu Manathattai

Project Repository: <https://github.com/Ashish-Basetty/StreamBed>

Abstract

StreamBed is a distributed machine learning deployment and inference system. It is designed to be adaptable to diverse network and computational bottlenecks, focused on an edge-server streaming architecture. Through containerization and a custom protocol, StreamBed allows for concurrent edge and server inference on perception embeddings, with support for training and evaluation and custom feedback signaling. Features include a TTL adaptable local frame storage, QUIC-based streaming, health monitoring and restarts, and dataflow monitoring. A dynamic routing system minimizes system downtime across a cluster of devices, and a userfriendly management portal allows for centralized monitoring of a heterogeneous computing system.

The system has been extensively tested through deployed cloud GCP [1] infrastructure, including under network throttling conditions and server inference slowdown. Chunk throttling improves embedding delivery from 60% to 98%, while JPEG compression combined with throttling achieves 100% frame delivery at 4.9 FPS for a 16 $\times$ improvement over unoptimized raw frame streaming. A sidecar extended from the open-source quic-go [2] package allows for low-latency and reliable streaming, integrated with deployment services and an internal routing/DNS system. To simulate realistic use, StreamBed uses a distributed PPO model trained on the Crafter [3] environment, as is done in previous research [4]. StreamBed’s test implementation uses a strong teacher and lighter student to maximize performance with ample server compute resources, while still maintaining sufficient student performance under restricted network/compute conditions. Teacher and student action logits are combined using an exponential staleness heuristic, to provide a reward boost of 147% against pure student logits with high feedback, and a score boost up until a reply delay of 38 ms.

1 Introduction

Video surveillance and analytics systems constantly generate massive data volumes. A single 1080p camera operating at 30 fps produces approximately 180GB of raw footage per day. State-of-the-art machine learning models, such as OpenVLA’s 7B parameter model [5], require expensive compute resources (Nvidia RTX 4090, with 7+ GB Memory). On the other hand, running ML inference exclusively in the cloud introduces unacceptable round-trip latency ($> 100\text{ms}$) for real-time applications such as autonomous vehicles, security monitoring, and industrial inspection. The central thesis of StreamBed is integrating the benefits of cloud inference at scale with the low-latency requirements of edge applications, while providing a full framework for a heterogeneous cluster with varying network availability.

Current VLA architectures are designed around a unified model processing raw video data, despite the high network cost of maintaining multiple constant live video streams. StreamBed provides a system for streaming lighter embeddings in tandem with sparser video frames, allowing full frame data to be retrievable on-demand and server-side planning with full data. The lightweight protocol, routing, and deployment systems allow users to focus on model refinement and evaluation while minimizing downtime and troubleshooting. The goal of this work is to explore system-level design tradeoffs in distributed machine learning deployment under real-world constraints.

1.1 Core Developments

This project makes the following contributions:

1. Presents the design and implementation of StreamBed, a distributed edge-server inference system that supports concurrent local and remote execution over shared perception embeddings.
2. Introduces a lightweight streaming and routing framework built on persistent transport connections with adaptive bandwidth control and failure-aware rerouting.
3. It develops a resource-aware local storage mechanism that dynamically manages high-fidelity frame retention under device constraints.
4. It provides an empirical evaluation demonstrating how system-level adaptations, such as throttling, compression, and distributed inference, affect throughput, latency, and downstream decision performance in latency-sensitive environments.

1.2 Major Features

- Centralized deployment and model management system, with multi-cluster search and advanced health monitoring
- Clear and unified device management portal, with complete stream state, routing, and health observability
- Dynamic TTL local frame storage that maintains cache based on live disk-pressure
- Quic-go derived networking sidecar, that minimizes downtime on rerouting, minimizes processing latency, and automatically adjusts frame-embedding interpolation based on system demands
- Dynamic frame-embedding interpolation system, monitoring network/server pressure signals to maximize use of available bandwidth
- Idempotent deployment management and automatic restart system for device failures
- Distributed embedding generation pipeline based on MobileNetV2 to stream features
- Crafter environment teacher-student inference system based on SOTA PPO methods, dynamically boosting a light local model's performance
- Full automated test suite, including component tests, docker-based integration tests, and provisioned cloud test infrastructure to validate cross-device functionality

2 Background and Related Work

2.1 Edge Device Inference

Modern vision-based machine learning systems are applied across various industries, including robotics and smart camera systems. These systems need to balance reactivity with task accuracy, which has often been handled with local inference compute. However, new architectures have improved performance at the cost of greater resource requirements, and assume full data transmission when outputting actions. This provides a potential challenge in adapting models to small remote devices running without expensive hardware.

2.2 Apache Kafka

One popular industry system for similar data stream processing tasks is Apache Kafka [6]. Kafka is designed around a broker-mediated distributed producer-consumer architecture [7], mapping *events* to *topics*, whose processing is logically divided into *partitions*.

However, the latency minimization required for a perception model introduces a different architectural priority. Since data can be cleanly partitioned onto relatively independent sensor streams, and because many models store past state in tensor form, partition remapping should be minimal and has heavy additional constraints. In addition, by working with long-lived transport *streams* rather than log-based single events, a distributed inference system can reduce complexity and overhead from the broker system. Finally, because embedding data is sent at a low-variance rate per-producer, and with relatively low variation in byte count, dealing with hot-keys and load balancing can be made more minimal.

To account for this, StreamBed minimizes remapping, routes logical streams, and allows for asynchronous rerouting. A central system is used for both health monitoring and routing, to minimize system downtime. Due to the heavy tensor stored state nature per stream required for models, StreamBed preserves temporal continuity and reduces tensor representation migration across consumers. StreamBed also uses persistent QUIC connections to reduce latency at the cost of potential throughput from batch reads from the brokers.

2.3 Zookeeper

Apache Zookeeper [8] is a distributed coordination system, operating on a hierarchical tree design with ephemeral nodes tied to processes. It is designed around high reliability and notifications to support service discovery, configuration management, and failure detection in distributed systems. StreamBed provides similar centralized health monitoring and discovery functionality, with additional support for ML model deployment and management.

2.4 Container Registry

StreamBed uses the DockerHub [9] container repository service to allow anyone to easily push inference containers that abide by a specific contract. This is integrated with a control plane designed around a cluster controller and device daemons to make standardized deployments possible. The StreamBed discovery service goes beyond Zookeeper to include the device setup environment.

2.5 Video Compression

Another challenge associated with perception streaming in spotty networks is minimizing size per frame. In video streaming applications, this problem has often been addressed through adaptive compression [10]. Other work in the machine learning space has identified major performance losses associated with classical compression [11] on Deep Neural Network performance.

Research has been done on adjusting compression rates to minimize performance loss, but past solutions have added significant extra computational complexity, and require task-specific training [12]. StreamBed is built around transmitting the tensor output of initial perception layers in machine learning models. By delegating computation to edge devices, semantic details are preserved without requiring a complex compression heuristic, and can allow a lighter and heavier model to run on the same perception output in tandem on edge and server respectively.

3 System Design

The StreamBed system extends a prototype system built for UCLA’s graduate Big Data Systems course, including a MobileNet embedding generator system developed by Ivan Sit, and an initial UDP interleaving protocol developed by Vishnu and Fred. The TTL frame store and API was also developed by Vishnu Manathattai and Fred Zhang. Further features and the control plane prototype were built by the author in this extension.

3.1 Architecture Overview

Figure 1 shows the high-level data and control flows. Each device has several microservices to prevent service failure from propagating, and dividing processes into monitoring/deployment, inference, and networking/streaming management. A separate, scalable controller is reachable via a basic user interface, and functions as a single source of truth for cluster state.

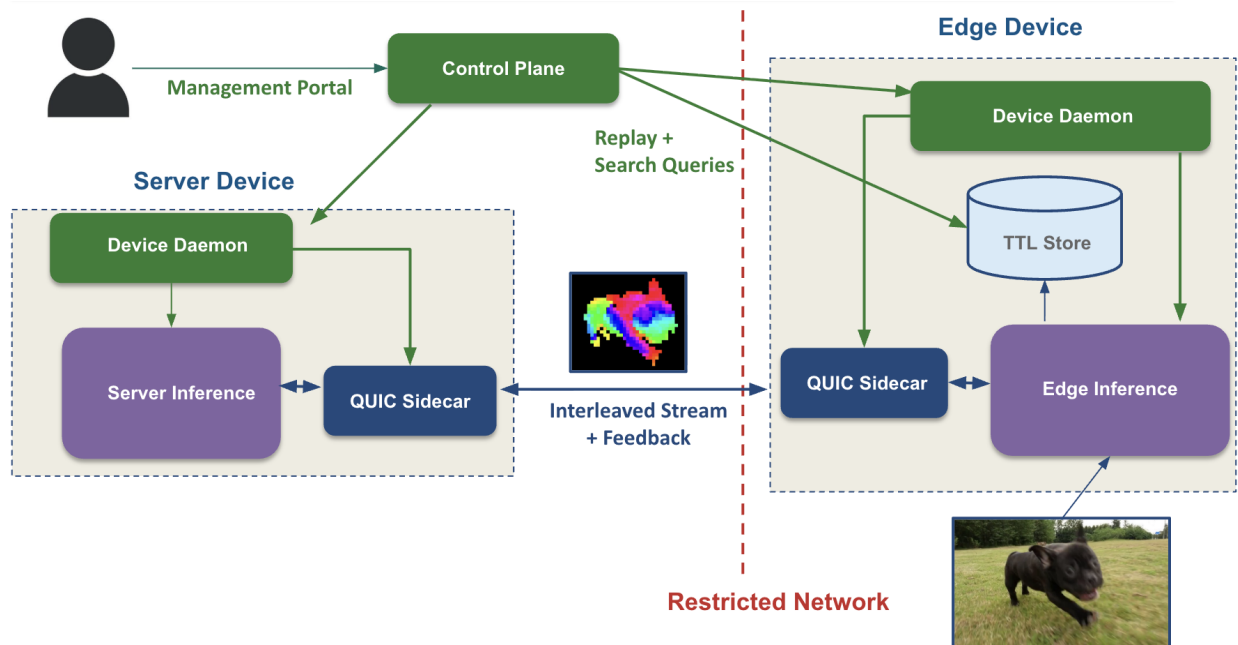


Figure 1: StreamBed system architecture overview. Data flows in blue, control flows in green.

3.2 Local Frame Storage

3.2.1 TTL Store Design

The `TTLManager` monitors disk usage via `shutil.disk_usage()` and computes TTL using linear interpolation:

$$\text{scale} = \frac{\text{free\%} - \text{critical\%}}{1.0 - \text{critical\%}}, \quad \text{TTL} = \text{TTL}_{\min} + \text{scale} \times (\text{TTL}_{\max} - \text{TTL}_{\min}) \quad (1)$$

Each node uses a SQLite metadata database to allow for frame/embedding queries marked by timestamp. When remaining disk space drops below 10%, TTL collapses to 30 seconds to prevent overflow. A periodic cleanup loop runs every 60 seconds to identify and delete expired entries from the local database and disk.

An API system allows for user query of frames from a specified timestamp range, allowing recent full video data to be retrieved for evaluation/training. Due to varying space constraints on different edge devices, there is no explicit storage guarantee, but the TTL system maximizes usage of available disk.

Free Disk Space	Scale Factor	TTL (seconds)
90%+	≈ 0.89	$\approx 3,200$ (53 min)
50%	≈ 0.44	$\approx 1,615$ (27 min)
< 10%	0.0	30

Table 1: TTL values at representative disk usage levels.

3.3 Streaming Sidecar System

3.3.1 Logical Chunking

Each `StreamFrame` is serialized with a struct-packed binary header including model version string, source device ID string, frame data, and embedding data. Each chunk is prefixed with a `Tag` and metadata. The `Tag` separates chunks in the StreamBed system into three logical groupings: `KindLossyData`, `KindLosslessData`, and `KindControl`.

Under lossy network conditions, `KindLossyData` is prioritized for dropping in Edge-to-Server streaming. A server-to-edge stream returns `KindControl` throughput feedback and action data.

3.3.2 QUIC Protocol

StreamBed’s streaming is built atop quic-go [2], separate from the control plane daemon and inference containers. A duplex UDP connection is used for data flow with the inference container, and feedback signals are used with decay to periodically calculate a target bit transmission rate, which is then used to calculate the drop rate of `KindLossyData` chunks.

3.3.3 Routing

Automatic system rerouting or manual routing updates are pushed to a daemon running on each device using a `PUT /stream-target` API call. These updates are picked up by the Go sidecar using periodic `GET /stream-target` polling of the device daemon, which then generates a new QUIC connection to the specified target via `ip:port` on target update.

3.3.4 Bandwidth Estimation

The bandwidth estimation algorithm uses an application-layer implementation of an AIMD algorithm [13]. In this setup, a slow additive increase allows the bandwidth estimate to slowly rise over time, dropping by a multiple when received throughput falls below 70% of sent throughput.

The additive increment rate is set to $\Delta = 400$ kbps, and decrement update is $R_{new} \leftarrow 0.7R_{old}$. Throughput rate is approximated by the below heuristic:

$$R_d = \frac{R_{received}}{R_{sent}} \quad (2)$$

where R_{sent} and $R_{received}$ are the sent and received throughputs over the sampling window. $R_{received}$ is transmitted through specially tagged FBCK packets. To account for temporary jitter, an exponentially weighted moving average (EWMA) is used to update the feedback rate R_{FBCK} on the server end:

$$R_{FBCK}^n \leftarrow \alpha R_{window} + (1 - \alpha) R_{FBCK}^{n-1} \quad (3)$$

R_{window} represents bits received in the last window period. In this implementation, $\alpha = 0.5$, and the rate window is 2 seconds.

3.4 Control Plane

The StreamBed control plane allows for centralized management of model deployments, container health monitoring, stream monitoring, metrics, and routing across an entire **cluster**. Devices in the system can register as **edge** or **server** instances, and can be managed remotely. Interaction with the control plane on the device's end is done through a StreamBed **daemon**, which communicates with a dedicated **controller** instance.

Devices are grouped logically into **clusters**, so that one cluster can be mapped to one controller, meaning management/monitoring can be done through an API call to only one service.

3.4.1 Routing and Registration

New devices send the controller a `POST /register` request. A scheduled process then automatically routes unassigned edges onto an active server with the lowest number of concurrent connections.

3.4.2 Health Monitoring

Devices can have a health **status** of **Active**, **Unresponsive**, or **Unknown** (upon recent deployment). There can also be a **data_flow_state** of **flowing**, **idle**, or **unknown**.

Heartbeats are automatically sent by the Go sidecar through the **sidecar-heartbeat**, which updates a central health table. Heartbeats also contain information on routing ports/ip of devices, and are coalesced with deployment information for the monitoring frontend.

Automatic failure detection triggers when the sidecar has a dead container heartbeat or no heartbeat. The controller attempts to tear down the preexisting container and redeploy with exponential backoff up to a maximum number of attempts. If the container cannot be restarted, a server's edges are marked as unrouted, and automatically routed to an active server.

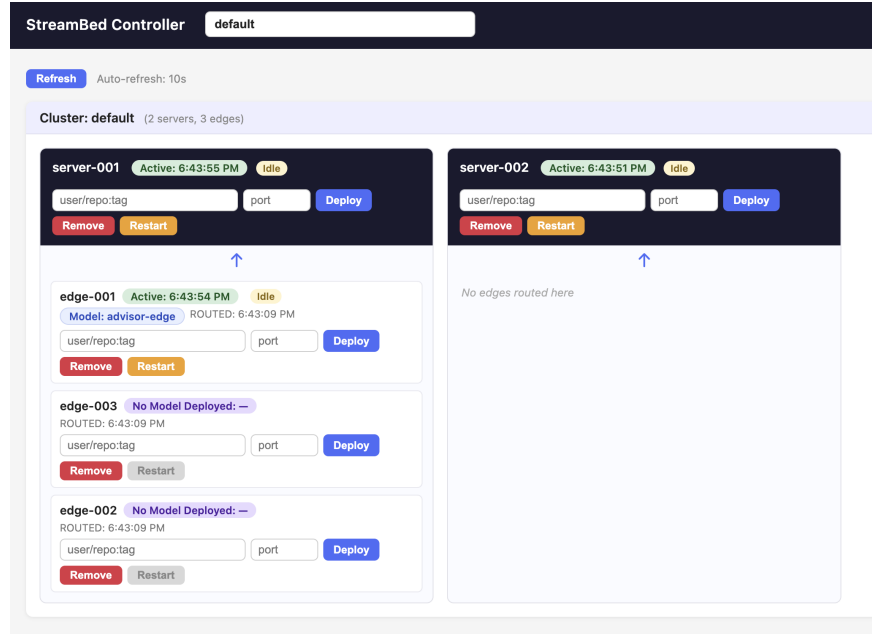


Figure 2: StreamBed management portal, showing registered devices, health monitoring, model deployments, and routing

3.4.3 Deployments and Monitoring

Deployments are managed through the `POST /deploy`, and `DELETE /delete` controller endpoints. The controller also serves a simple HTML and JS page to make these functions simple. The controller makes use of a `PUT /stream-target` endpoint on the daemons in order to propagate routing info periodically.

3.5 Local and Cloud Deployment

Local integration tests use a simple Docker Compose, deploying six services (3 edge services, 2 server services, controller), as well as associated daemons and QUIC sidecars. Local data directories are mounted for tests and persistence.

To simulate real devices spread across a network, GCP [1] test infrastructure has been provisioned, using Terraform and OpenTofu [14] for declarative and reproducible deployment. Tests have been set up for this infrastructure to validate functionality, and will be expanded further. Five virtual machines have been provisioned, including a controller node and two server and edge nodes.

4 Evaluation and Results

4.1 Test Suite

We maintain 72 unit tests and additional integration tests covering all system components:

Component	Tests	Coverage
Inference Pipeline	12	Model loading, embedding shape, batch inference
FrameStore+TTL Manager	21	SQLite metadata, Disk Pressure, Retrieval
Retrieval API	12	Health, frames, embeddings, error handling
Stream Protocol	13	Chunking, codec, UDP split, custom payloads (CSTL/CSTR)
Heartbeat & State DB	12	Liveness, persistence, recovery
Controller Routing	2	Routing table cleanup on disconnect
Python Unit Total	72	All passing
Sidecar Bandwidth (Go)	5	Composite min, fallback, sampling bounds
Sidecar Policy (Go)	9	Rate-limit, control/embed protection, CSTL/CSTR drop
Sidecar Protocol (Go)	1	Prefix classification
Go Unit Total	15	All passing
Integration (router/storage)	29	End-to-end delivery, routing, failure detection
Integration (QUIC)	8	Reverse path, rerouting, chaos matrix, fairness, codec
Integration (GCP cross-VM)	5	Handshake, registration, startup, ACTN latency
Integration Total	42	All passing

Table 2: Test suite breakdown

4.2 Embedding Throughput Validation

For the first experiment, we designed a base throughput benchmark, which sends pre-computed frames through a Docker proxy simulating an unstable network to the server container. This validation was to confirm size reduction with MobileNetV2 embeddings, evaluate frame throughput on a lossy network, and measure average latency reduction with reduced frame size.

We tested two data modes (**embeddings** and **raw frames**) under three network simulation configurations. The benchmark measures FPS, delivery rate, and latency under three network conditions.

Mode	Condition	Baseline			Throttling only			Throttling + JPEG		
		FPS	Delivery	Latency	FPS	Delivery	Latency	FPS	Delivery	Latency
embeddings	clean	9.4	60.0%	103ms	14.8	98.0%	64ms	15.4	100.0%	62ms
embeddings	50ms delay	6.7	56.0%	145ms	11.8	100.0%	82ms	11.5	98.0%	83ms
embeddings	10% loss	9.6	62.0%	100ms	13.9	90.0%	68ms	13.9	90.0%	68ms
raw_frames	clean	0.3	2.0%	3324ms	2.2	22.0%	276ms	4.9	100.0%	200ms
raw_frames	50ms delay	0.3	2.0%	3244ms	2.0	24.0%	337ms	6.7	100.0%	144ms
raw_frames	10% loss	0.3	2.0%	3275ms	0.4	4.0%	1518ms	4.2	94.0%	231ms

Table 3: Throughput benchmark results (50 frames per run)

Adding a 1ms pacing delay between chunk transmissions improved embedding delivery from 60% to 98% under clean conditions, indicating that a lack of flow control caused silent drops in our network simulation container. Throttling controls the send rate, giving the receiver time to drain its buffer between packets. While JPEG compression and throttling improved delivery success, this introduces a delay in response and incomplete information being transmitted to the inference server.

This validates the value of StreamBed in reducing bitrate to avoid overwhelming the network. StreamBed’s embedding format minimizes information loss and latency under unideal network conditions.

4.3 Crafter Environment Teacher-Student Joint Inference

As a test in functionality, a deep reinforcement learning agent was trained on the Crafter [3] environment. This environment aims to emulate a Minecraft-like game in 2-dimensions, with players needing to ”forage for food and water, find shelter to sleep, defend against monsters, collect materials, and build tools”. The environment is set up to offer relatively sparse rewards (+1 for achievements, and ± 0.1 for lost/regenerated health points). The environment was chosen due to its simple OpenAI Gym [15] compatible setup, relatively simple 16 keyboard input action space, and focus on higher level planning in tandem with simpler action sequences.



Figure 3: Crafter environment capture from trained agent trajectory

Previous research has involved training an agent [4] to successfully produce high-reward trajectories. Referencing their training methodology, a 1.66 million parameter PPO teacher model was trained on 5 million steps, to achieve a mean test reward score of 10.39.

Next, the three initial convolutional layers and subsequent linear layer of the teacher model were frozen. From this encoder-like output, a 2000 parameter student head was trained from the Crafter environment, to have a mean test reward of 2.57.

Both models were containerized, and deployed on the StreamBed system. The EMBD packets transmitted were the outputs of the frozen first four layers, which were processed simultaneously by the smaller student edge model and larger teacher server model. A cadence schedule dictated the reply rate of the server model, with teacher action logits transmitted back through StreamBed, and combined on the edge end with the below formula:

$$l_{final} = w \cdot l_{teacher} + (1 - w) \cdot l_{student}, w = e^{(-age/\tau)}, \tau = 0.05s \quad (4)$$

In this formula, teacher logits are given a decaying importance based on their staleness, preventing stale logits from affecting later actions significantly.

By adjusting cadence (teacher reply every N embeddings sent), it was found that the teacher model’s input significantly boosted reward, while still maintaining a reward boost up to a delay of 38 ms.

Cadence	Score	Mean Advice Age	Boost vs. Edge Only
1	6.59	7 ms	+3.92
5	5.70	14 ms	+3.03
10	2.72	38 ms	+0.05
20	1.90	57 ms	−0.77 (hurts)
∞ (edge only)	2.67	—	0

Table 4: Cadence sweep on StreamBed system, 10 eval episodes per step

This result showed that not only is a distributed inference system effective, but can also trade off server load with reply latency and reward. While resource constraints prevented full testing of a VLA server system, the combination of higher-level planning with lighter models running at lower latency may overcome the shortcomings of either system independently.

4.4 Constrained Network Evaluation

To test the live function of the bandwidth estimator and the adaptive frame streaming, this experiment introduced a network simulator UDP proxy through which all traffic was routed. It uses a bucket system, where a bucket is decremented when bits are forwarded, and slowly refilled over time at a set bitrate. If the bucket is empty, packets are dropped. This is designed to mimic a real network with a constrained throughput, such that packets are dropped randomly if throughput is exceeded.

The proxy was set to a schedule, where the allowed bitrate (`throttle_bps`) would change every 60 seconds. The StreamBed system successfully throttled frame packets when necessary and maximized frame streaming when possible. Due to the bandwidth estimator design, there was a delay of between 5 and 40 seconds after a significant `throttle_bps` change before the total sent packet bitrate (`embd+chnk`) reached equilibrium.

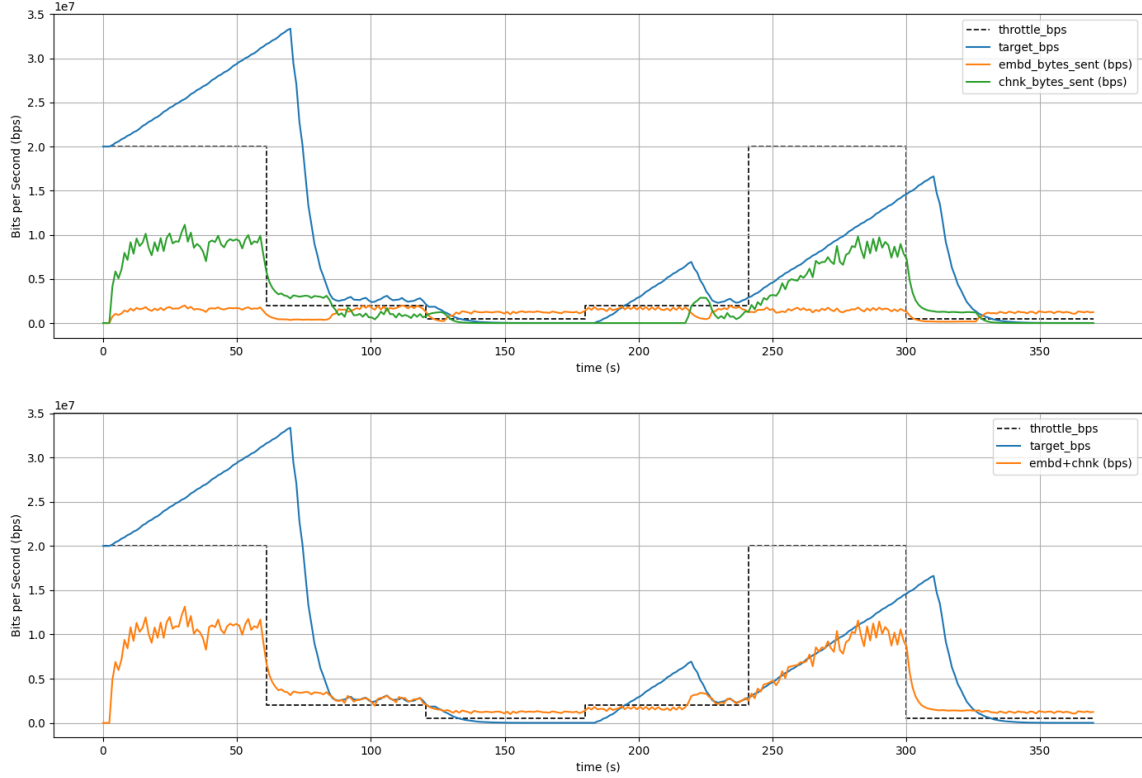


Figure 4: Proxy Rate Limit, Target Bitrate, and Smoothed Sent Bitrates during Experiment

The internally calculated target bitrate `target_bps` was logged through metrics. This is a system estimate on the sent bitrate upper bound, above which significant packet loss occurs. This sometimes exceeded the `throttle_bps`, as no packets were dropped when produced data was lower than `throttle_bps`. The lower graph shows that a decrease in `throttle_bps` led to a multiplicative decrease in `target_bps`, and an increase in `throttle_bps` led to a slower additive `target_bps` rise.

Crucially, the experiment confirmed that StreamBed successfully prioritizes dropping frame packets, and that this maintains a steady stream of embeddings even under constrained network conditions. The upper graph represents embeddings sent as `embd_bytes_sent`, and frames sent as `chnk_bytes_sent`. After equilibrium is reached, the frame packets sent drop enough to allow full throughput of the embedding packets.

We can examine the behavior at the `throttle_bps` transitions. When `throttle_bps` goes from 2,000,000 bps to 200,000 bps, both sent frames and embeddings drop as the proxy randomly drops excess packets. `target_bps` reacts within 25 s, and subsequently `chnk_bytes_sent` drops as the edge sidecar preemptively drops frame packets. When `throttle_bps` goes to 50,000 bps, initially all packets are dropped by the proxy, but then the edge sidecar further drops frame packets. The additive increase of `target_bps` as `throttle_bps` rises allows more and more frames to be sent, with equilibrium `embd_bytes_sent` maximized under all network conditions.

5 Limitations and Future Work

Significant progress has been made towards building StreamBed to be deployment ready. The following areas are strong candidates for further development:

1. Full VLA edge-server testing, including with physical perception and actions
2. Improved security/authentication protocol for device registration
3. Improved horizontal scaling and distributed router system to handle request load
4. Horizontal scalability of controller, including logical cluster-to-controller mapping algorithm
5. Improved control plane redundancy and failover procedures
6. Plug-and-play prebuilt containers for popular encoder models (ex. CLIP [16], DINOv3 [17])

StreamBed targets a specific niche of applications: devices with basic hardware acceleration but unreliable connectivity. Edge devices on well-connected networks or with ample compute resources may be better served by simpler architectures.

6 Conclusion

StreamBed demonstrates that distributed edge-to-server inference on live systems from perception data is feasible and effective. As edge ML and robotics applications scale beyond the lab, their unique needs require distributed systems innovation, focusing on latency reduction and adaptability to the unreliable environment outside of a datacenter.

Extensive testing in both local and cloud deployments validates that the system is robust and can be extended to production environments. Embedding transmission has been clearly shown to reduce bandwidth requirements, and a distributed edge-server inference model is shown to provide a significant boost against edge inference alone. Full pipeline network testing supports that StreamBed maximizes bandwidth usage, effectively prioritizing embeddings and dropping full frames as necessary.

The StreamBed project has been listed under the MIT License and is public on GitHub. Rather than serving as a unified service for customers, the project is designed to function as a baseline or framework for similar ML serving systems, and as an exploratory experiment in how they might be designed.

References

- [1] Google. Google Cloud Platform. <https://cloud.google.com/>. Accessed: 2026-05-21.
- [2] The quic-go authors. quic-go: A QUIC implementation in pure Go. <https://github.com/quic-go/quic-go>. Accessed: 2026-05-21.
- [3] D. Hafner. Crafter: Benchmarking the spectrum of agent capabilities. <https://github.com/danijar/crafter>. Accessed: 2026-05-21.
- [4] A. Stanić, Y. Tang, D. Ha, and J. Schmidhuber. Learning to generalize with object-centric agents in the open world survival game Crafter. *arXiv preprint arXiv:2208.03374*, 2022.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

- [6] Apache Software Foundation. Apache Kafka. <https://kafka.apache.org/>. Accessed: 2026-05-21.
- [7] J. Kreps, N. Narkhede, and J. Rao. Kafka: A distributed messaging system for log processing. In *Proc. NetDB Workshop*, 2011.
- [8] Apache Software Foundation. ZooKeeper: A distributed coordination service for distributed applications. <https://zookeeper.apache.org/doc/r3.5.1-alpha/zookeeperOver.html>. Accessed: 2026-05-21.
- [9] Docker, Inc. Docker Hub documentation. <https://docs.docker.com/docker-hub>. Accessed: 2026-05-21.
- [10] T. Stockhammer. Dynamic adaptive streaming over HTTP: Standards and design principles. In *Proc. 2nd Annual ACM Conf. on Multimedia Systems (MMSys '11)*, pages 133–144. ACM, 2011. <https://doi.org/10.1145/1943552.1943572>.
- [11] M. Poyser, A. Atapour-Abarghouei, and T. P. Breckon. On the impact of lossy image and video compression on the performance of deep convolutional neural network architectures. *arXiv preprint arXiv:2007.14314*, 2020.
- [12] T. Murad, A. Nguyen, and Z. Yan. DAO: Dynamic adaptive offloading for video analytics. In *Proc. 30th ACM Int. Conf. on Multimedia (MM '22)*. ACM, 2022. <https://doi.org/10.1145/3503161.3548249>.
- [13] Y. R. Yang and S. S. Lam. General AIMD congestion control. In *Proc. 8th Int. Conf. on Network Protocols (ICNP)*, Osaka, Japan, 2000. IEEE.
- [14] The OpenTofu Authors. OpenTofu: The open source infrastructure as code tool. <https://opentofu.org/>. Accessed: 2026-05-21.
- [15] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. <https://github.com/openai/gym>, 2016.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proc. 38th Int. Conf. on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 8748–8763, 2021.
- [17] O. Siméoni, H. V. Vo, M. Seitzer, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025.